Appendix: AI-Assisted Coding Guide

Using Cursor (Free Plan) for This Test

This guide helps you use Cursor IDE efficiently with minimal token usage while completing this assessment. AI assistance is allowed and encouraged – it's a real-world skill!

What is Cursor?

Cursor is a VS Code-based IDE with built-in AI coding assistance. The free plan gives you limited requests, so use them wisely.

Installation

1. Download Cursor from https://cursor.com
2. Install and sign up for a free account
3. Open your project folder in Cursor

Token-Efficient AI Prompting Strategy

Golden Rules for Minimal Token Usage

1. **Be specific and concise** – Vague prompts waste tokens on clarification
2. **Ask one thing at a time** – Don't bundle multiple questions
3. **Provide context upfront** – Include relevant code snippets
4. **Use Tab completion** – Free and doesn't use chat tokens
5. **Learn from responses** – Don't ask the same thing twice

Recommended Workflow

Phase 1: Setup (Use Commands, Not AI)

Don't waste AI tokens on setup – just follow these commands:

```
# These are standard commands - no AI needed
curl -LsSf https://astral.sh/uv/install.sh | sh
mkdir ekantipur-scraper && cd ekantipur-scraper
uv init
uv add playwright
uv run playwright install chromium
```

Phase 2: Understand the Website Structure (Manual Work)

Before asking AI anything:

1. Open https://ekantipur.com in your browser
2. Press F12 to open DevTools
3. Use the Element Inspector (Ctrl+Shift+C) to find:
    - Entertainment section link/navigation
    - News article containers (class names, structure)
    - Cartoon section location
4. **Write down the selectors you find** – This becomes YOUR input to the AI

Example notes you might take:

```
Entertainment link: nav item with text "मनोरञ्जन"
Article cards: div.article or similar
Each card has: img tag, h2 for title, span.author
Cartoon section: look for "व्यंग्यचित्र" text
```

What NOT to Ask AI (Token Wasters)

❌ Bad Prompt	Why It Wastes Tokens	✅ Better Alternative
"How do I scrape a website with Playwright?"	Too broad, generic tutorial	Read official docs first
"Write the complete scraper for ekantipur.com"	Uses 500+ tokens, you learn nothing	Ask for skeleton, then specific parts
"What's wrong with my code?" (no code shown)	AI will ask for code, wasting a round	Always include the relevant code
"Explain Playwright to me"	Use docs instead	https://playwright.dev/python/docs
"How do I install uv?"	It's in this document	Just follow the setup commands

---

Free Features to Maximize

1. Tab Autocomplete (Unlimited & Free)

Start typing and press Tab for completions. Works great for:

- Variable names
- Method names after `.`
- Common patterns
- Import statements

2. Cursor's Cmd+K / Ctrl+K (Inline Edit)

- Select code and press Cmd+K (Mac) or Ctrl+K (Windows)
- Ask for small, focused modifications
- More token-efficient than chat for edits

Example: Select a function, press Ctrl+K, type:

```
Add try/except for None elements
```

3. Hover Documentation

- Hover over Playwright methods for quick docs
- Use Go to Definition (F12) to understand code
- No tokens used!

Debugging Without AI (Save Tokens!)

Try these before asking AI:

```python
# Print element count
cards = page.query_selector_all(".article-item")
print(f"Found {len(cards)} cards")

# Print HTML to see structure
first_card = page.query_selector(".article-item")
print(first_card.inner_html())

# Check if selector exists
element = page.query_selector(".your-selector")
print(f"Element found: {element is not None}")

# Pause and inspect manually
page.pause()  # Opens Playwright Inspector!

# Screenshot for debugging
page.screenshot(path="debug.png")
```

Quick Reference Card

Task	Efficient Approach	Token Cost
Setup & install	Follow docs/commands	0
Find selectors	Browser DevTools (F12)	0
Script structure	One prompt for skeleton	~60
Extraction logic	One prompt with HTML example	~50
Error handling	Include error + code snippet	~40
Debugging	Use print() and page.pause()	0
Understanding code	Read Playwright docs	0

Resources (Use Before AI)

- **Playwright Python Docs:** https://playwright.dev/python/docs/intro
- **Playwright Selectors:** https://playwright.dev/python/docs/selectors
- **Playwright Locators:** https://playwright.dev/python/docs/locators
- **uv Documentation:** https://docs.astral.sh/uv/
- **Python JSON module:** https://docs.python.org/3/library/json.html

---

Final Notes

Using AI effectively is a skill we value at Audio Bee. This guide teaches you to:

1. **Do the easy stuff yourself** – Setup, DevTools inspection, basic debugging
2. **Ask smart questions** – Specific, include code context, one thing at a time
3. **Learn from each response** – Don't repeat similar questions
4. **Maximize free features** – Tab completion, Ctrl+K edits, hover docs

The goal isn't to avoid AI – it's to use it like a professional: efficiently, strategically, and as a force multiplier for your own skills.